

MediFetch — Full Package

All-in-one blueprint: **Hospital wireframes (Figma-ready page list)**, **Patient mobile flow**, **API spec**, **Database & unique ID design**, and a **1-page slide** for your project defense. This is ready to copy into Figma, Notion, or into your dev repo.

1) Figma-ready Page List & Wireframe Notes — Hospital / Doctor Dashboard

Top-level pages (hospital web dashboard)

1. Login / OTP verification

2. Fields: email/phone, password, OTP entry.

3. Micro: 'Switch to Patient View' toggle.

4. Onboarding / Setup

5. Hospital profile (logo, address, departments), add initial doctors, set hospital code.

6. Hospital Dashboard (Home)

7. Widgets: Recent uploads (last 24h), Unread messages, Urgent flags, Pending share requests, Quick patient search (global), Analytics card (uploads/day).

8. CTA: Upload Report, Create Prescription, Add Patient

9. Patient Directory / Search

10. Search bar (name, patientID, phone), filters (date range, department, tag), result list (photo, name, patientID, last visit, quick actions).

11. Patient Profile (Doctor view)

12. Header: Name, patientID, DOB, blood group, alert tags (allergy), emergency contact.

13. Tabs: Timeline (chronological), Reports (filter by type), Prescriptions, Shared With, Appointments.

14. Right rail: Quick actions (Upload report, Create prescription, Message patient, Share snapshot)

15. Upload Report Page / Modal

16. Form: Patient selector, report type (lab/x-ray/prescription/etc.), date, clinician notes, file upload (drag/drop), visibility (private / shared with X hospitals), tags, consent checkbox.

17. Progress bar & preview thumbnail.

18. Report Viewer

19. Full-screen PDF/Image viewer, metadata panel (uploaded by, date, tags), comment thread, version history, download (signed URL), share button.

20. Message Center (Inbox)

21. List of threads (unread badge), thread view on the right with attachments, quick reply, mark urgent flag.

22. Incoming Shares / Requests

23. Requests list from patients/other hospitals (approve/decline), reason field, duration of access.

24. Admin Console

- Manage doctors, roles, hospital settings, access logs, generate/export audit reports.

25. Settings

- Notification preferences, API keys (if any), storage usage, security settings.

Wireframe notes - Use a 12-column grid. Left nav (collapsed) with icons for Dashboard, Patients, Upload, Messages, Requests, Admin. - Keep the patient timeline central — that's the primary focus for doctors. - Add micro-interactions: upload progress, toast notifications, confirmation modal on destructive actions.

2) Patient Mobile Flow — Screen-by-screen (mobile-first)

Goal: signup → view report → share with another hospital

Flow & Screens

1. Welcome / Onboarding

2. CTA: Sign up / Login. Short explainer: "Your secure medical vault. Doctors upload results here."

3. Sign up / Sign in (OTP)

4. Fields: name, phone, email (optional), DOB. Create password or OTP flow.

5. After signup: generate unique **MediFetch ID** (displayed as primary key, example:

MF-2025-0A4B9C

)

6. Profile Setup (quick)

7. Emergency contact, allergies, blood group, chronic conditions. Option to upload a profile photo.

8. Home / Timeline

9. Chronological feed of records (cards) — date, type, uploaded by (Dr. X), hospital name, download icon, share icon.

10. Top CTA: Upload personal file

11. Emergency Snapshot button (floating)

12. Report Detail

13. Viewer for PDFs/images, metadata (uploader, date), notes from doctor, share button, download, request review.

14. Upload Personal Document

15. Camera capture or file pick, type tag, optional note, submit.

16. Share Record Flow

17. Step 1: Select records (multi-select)

18. Step 2: Choose recipient type — *Existing hospital* / *New hospital via MediFetch ID* / *Doctor's email/phone*

19. Step 3: Set access duration & permissions (view / comment / download) + Consent checkbox

20. Step 4: Confirm & Send → shows success and logs the share in history

21. Link to Hospital (using MediFetch ID)

22. Input hospital code or use hospital search. Alternatively: enter hospital's system to request linking — hospital verifies patientID and approves link.

23. UI: show linked hospitals list (and pending link requests)

24. Messages

25. Conversation list and individual threads. Attach reports directly in messages.

26. Settings / Sharing History

- Revoke access, export data, view access logs (who viewed which record & when), account security.

Mobile UX notes - Use big tappable targets and clear icons for download/share. - Show patientID prominently in profile (copy button). Emphasize: "Use this ID to register at partner hospitals." - Include confirmations and short toasts for privacy-critical actions (sharing, revoking).

3) API Spec (REST) — core endpoints (auth, records, share, messages, hospitals)

Auth

- `POST /api/auth/signup`
• Body: `{ name, phone, email?, password? }`
• Response: `{ userId, mediFetchId, token }`
- `POST /api/auth/login`
• Body: `{ phoneOrEmail, password }`
• Response: `{ token, user }`
- `POST /api/auth/otp/request`
• Body: `{ phone }` → sends OTP
- `POST /api/auth/otp/verify`
• Body: `{ phone, otp }` → Response `{ token, user }`

Patients & Linking

- `GET /api/patients/:patientId` (doctor/admin or patient themselves)
• Response: patient profile + linked hospitals list
- `POST /api/patients/:patientId/link-request`
• Body: `{ hospitalId, reason }` — creates a link request the hospital reviews
- `POST /api/patients/link-by-mediFetchId`
• Body: `{ mediFetchId, hospitalId }` — hospitals use this endpoint to attach an existing MediFetch user to their internal patient file (requires hospital approval)

Records

- `POST /api/records` (multipart/form-data)
• Auth: doctor/hospital or patient
• Body: `{ patientId, type, date, notes, visibility, tags } + file[]`
• Response: `{ recordId, status }`

- GET /api/records?patientId=&type=&from=&to=&tag=
- Returns list of record metadata (paginated)
- GET /api/records/:recordId
- Returns metadata + signed download URL (short-lived)
- DELETE /api/records/:recordId (soft-delete + versioning)

Sharing / Permissions

- POST /api/records/:recordId/share
- Body: { toUserId | toHospitalId, permissions: ['view', 'download', 'comment'], expiresAt? }
- Response: { shareId }
- POST /api/shares/:shareId/revoke
- Revokes access immediately
- GET /api/shares?patientId=
- Returns share history

Messages / Communication

- GET /api/threads?userId=
- Returns threads user participates in
- POST /api/threads (create thread)
- Body: { participants[], initialMessage }
- POST /api/threads/:threadId/messages
- Body: { senderId, content, attachments[] }

Hospitals

- GET /api/hospitals (search)
- GET /api/hospitals/:id (details)
- POST /api/hospitals/:id/approve-link (hospital admin approves patient link request)

Admin / Audit

- GET /api/audit?targetId=&action=&from=&to=
- Returns access logs (who viewed/downloaded/shared)

Auth & Headers - All protected endpoints require `Authorization: Bearer <JWT>` - Role checks performed server-side

Example: Upload record (curl)

```
curl -X POST "https://api.medifetch.app/api/records"
-H "Authorization: Bearer $TOKEN"
-F "patientId=123"
-F "type=lab"
-F "file=@/path/to/result.pdf"
```

4) Database Schema (core tables) + Unique MediFetch ID system

Core tables (relational)

- `users` (id PK, medi_fetch_id UNIQUE, name, phone, email, role, hashed_password, created_at)
- `patient_profiles` (id PK, user_id FK, dob, blood_group, allergies JSON, emergency_contact JSON)
- `hospitals` (id PK, name, code, address, contact)
- `doctors` (id PK, user_id FK, hospital_id FK, specialty)
- `records` (id PK, patient_id FK, uploaded_by FK, type, file_key, metadata JSON, encrypted_key, created_at, visibility)
- `shares` (id PK, record_id FK, from_user FK, to_user FK nullable, to_hospital FK nullable, permissions JSON, expires_at, revoked_at)
- `threads` & `messages` (for communication)
- `audit_logs` (id PK, user_id, action, target_type, target_id, ip, timestamp)
- `link_requests` (id PK, patient_id, hospital_id, status, requested_at, resolved_at)

Unique MediFetch ID design - Purpose: human-friendly primary key for patients that hospitals can use to lookup or link a patient (safe to expose). Not the DB PK. - **Format options:** - Short readable: `MF-YYYY-XXXXXX` (e.g., `MF-2025-A7B3C1`) - Or canonical: `MF-<base36(random6)>` or use UUID truncated. - **Generation rules:** - Generated at signup server-side. - Uniqueness enforced via DB unique index on `medi_fetch_id`. - Provide copy-to-clipboard and QR code in the profile (for quick linking at hospital desks).

Linking flow using MediFetch ID 1. Patient shows MediFetch ID or QR at hospital registration. 2. Hospital uses `POST /api/patients/link-by-mediFetchId` to request link. 3. System creates `link_requests` entry; hospital admin approves via `POST /api/hospitals/:id/approve-link`. 4. On approval, the hospital's local patient record stores `medi_fetch_id` and a `patient_link` is created. 5. If patient consents, hospital can request access to records (share request) or the patient can proactively share.

Indexing & performance - Index `records.patient_id`, `records.created_at`, `shares.to_user`, `shares.to_hospital`. - Use signed URLs & CDN for file delivery; store only metadata in DB.

Security - Store file encryption keys in a KMS (Key Management Service) and store per-file encrypted key in DB. - Keep PII minimized and encrypted at rest.

5) 1-Page Slide (Project Defense — single slide copy)

Title: MediFetch — Secure Medical Vault & Communication

Problem: Paper-based records and siloed hospital files cause lost information, delayed care, and repeated tests.

Solution (one line): MediFetch centralizes patient records, enables secure doctor-patient communication, and provides a portable MediFetch ID to link patients to hospitals instantly.

Key Features (bullets): - Encrypted medical record storage + audit logs - Doctor upload + patient timeline - Consent-based sharing & revoke - Secure messaging - MediFetch ID for cross-hospital linking

Tech & Security: React, Node/NestJS, PostgreSQL, S3 (encrypted), JWT + RBAC, signed URLs, KMS

Demo Flow (what you'll show): 1. Doctor uploads a lab result → 2. Patient receives notification & views record → 3. Patient copies MediFetch ID and links at a new hospital → 4. Patient shares record with new provider → 5. Emergency Snapshot visible

Impact / Metrics: - Faster diagnosis & fewer repeat tests - Better continuity of care and record portability

Ask / Next steps: Build MVP (auth, records, sharing, messaging) and test at one partner hospital.

6) Acceptance criteria & quick test cases

- Upload latency: record appears for patient within 60s of upload
 - Share & revoke: after revoke, recipient cannot download or view
 - Link by MediFetch ID: hospital can link and request access, patient receives link request
 - Audit logs: show user, action, target, timestamp for every view/download
-

7) Deliverables I can produce next (pick one)

- Figma wireframes: hospital dashboard + 6 main mobile screens (signup → view → share)
- React component skeletons (login, timeline, report viewer)
- OpenAPI (Swagger) spec for the API above
- One-page slide exported as PNG/PPTX

Final quick notes

- MediFetch ID is a *strong idea* — make it visible, copy-friendly, and QR-enabled for IRL hospital registration.
- Early priority: privacy + audit trails. Schools and hospitals will ask about compliance.
- If you want, I'll now generate the Figma page list as frames with component names (so you can drop it straight into Figma).

Document created by Ray — ready to start building. Pick which deliverable you want next and I'll make it.